

Funciones en JavaScript:

Funciones flecha

Funciones anónimas.

Puente Cognitivo

Para esta sesión es estrictamente necesario conocer y entender la **declaración de funciones regulares**, la correcta recepción de **parámetros** y el uso obligatorio de la instrucción **return** para devolver valores al hilo principal. Hoy cambiaremos la forma de escribirlas, pero la lógica de entrada (parámetros) y salida (retorno) sigue siendo matemáticamente idéntica.

Reto rápido

Analicen el siguiente bloque de código. Su tarea es identificar las tres partes fundamentales de la función regular y predecir qué imprimirá la consola.

```
function calcularDoble(numero) {  
  let resultado = numero * 2;  
  return resultado;  
}
```

```
let valorCalculado = calcularDoble(15);  
console.log(valorCalculado);
```

Situación

Imagina la logística de un restaurante de comida rápida. Tienes al Cocinero Principal. A este empleado le das un nombre, un uniforme y un contrato a largo plazo porque lo vas a llamar constantemente para que prepare hamburguesas. Esto equivale a una función regular con nombre (`function prepararHamburguesa()`).

Pero de repente, llega un camión con insumos y necesitas a alguien que simplemente baje las cajas en ese instante. No necesitas hacerle un contrato, ni darle un uniforme, ni saber su nombre. Solo le dices: "Tú, haz esto ahora mismo y luego vete". Esto es una función anónima.

En el desarrollo web moderno (y especialmente en frameworks como React), la pantalla está llena de botones y eventos. No queremos inventar 500 nombres de funciones para 500 botones distintos si la acción solo se usará una vez. Usamos funciones anónimas y, para escribirlas más rápido y con un comportamiento más predecible, utilizamos su evolución: las Funciones Flecha.

Hoy aprenderemos que las funciones son Valores. Al igual que `let edad = 25;` (donde el valor es un `Number`) o `let saludo = "Hola";` (donde el valor es un `String`), podemos hacer esto:

```
let miInstruccion = /*...una función aquí...*/;
```

Funciones como Ciudadanos de Primera Clase

En JavaScript, las funciones no son "ciudadanos de segunda clase" (como en otros lenguajes, donde solo pueden ser definidas y llamadas). Aquí, son "de primera clase". Esto significa que una función es un **VALOR**.

Como cualquier otro valor (un Number, un String, un Boolean), una función puede:

1. Ser asignada a una variable.
2. Ser pasada como argumento a otra función.
3. **Ser retornada** por otra función.

Expresión de Función (Function Expression)

Cuando aplicamos la regla #1 (asignar una función a una variable), creamos una **Expresión de Función**.

Las **Declaraciones** (function saludar() {})) son "elevadas" (hoisting). El motor de JS las "mueve" al principio del archivo. Puedes llamar a saludar() antes de haberla escrito en el código.

Las **Expresiones** (const decirAdios = ...) **NO** son elevadas. La variable decirAdios sigue las reglas de const: no puedes usarla antes de que sea declarada y asignada. Esto es, generalmente, un comportamiento más predecible y seguro.

Lo que sabíamos (Declaración):

```
function sumar(a, b) {  
  return a + b;  
}
```

Lo nuevo (Expresión):

```
const sumar = function(a, b) {  
  //      ^           ^  
  //      |           |  
  // Variable  Función Anónima (el valor)  
  // (const)   (fíjate que NO tiene nombre  
  //           después de 'function')  
  return a + b;  
}; // <-- ¡Punto y coma! Porque es el final de una asignación de variable.
```

Uso: Una vez guardada en la variable, la variable *se convierte* en la función. La llamamos usando el nombre de la variable. `let resultado = sumar(5, 10); // resultado es 15`

Pasar Funciones como Argumentos Podemos pasar una función anónima directamente como argumento a otra función que espera recibir una "instrucción".

Ejemplo con `addEventListener` (decirle a un botón qué hacer al ser presionado):

```
// Obtenemos el botón
const miBoton = document.getElementById("miBoton");

// Le pasamos una función anónima como segundo argumento.
// Esta función se ejecutará SÓLO cuando ocurra el 'click'.
miBoton.addEventListener("click", function() {
  console.log("¡Me hicieron clic!");
});
```

Pausa para cuestionar: ¿Vemos por qué no necesitamos un nombre?

La función solo existe para ser entregada a `addEventListener`. No la volveremos a llamar desde ningún otro lugar. Darle un nombre sería innecesario.

Funciones Flecha: La versión compacta y moderna

Introducidas en ES6 (ECMAScript 2015), las **funciones flecha** son una sintaxis más corta y elegante para escribir funciones, especialmente las anónimas. Son la forma preferida de escribir funciones en el JavaScript moderno, ya que eliminan la necesidad de las palabras clave `function` y `return` en muchos casos.

Las **Funciones Flecha** (`=>`) son *siempre* anónimas y se usan *siempre* como expresiones.

Analogía: La Evolución de un Mensaje

- **Declaración:** `function saludar() { ... }` (Un email formal).
- **Expresión:** `const saludar = function() { ... }` (Un mensaje de texto completo).
- **Flecha:** `const saludar = () => { ... }`

```
const sumarFlecha = (a, b) => { return a + b; };
```

Sintaxis Paso a Paso:

Empezamos con una Expresión de Función tradicional:

```
const sumar = function(a, b) {  
  return a + b;  
};
```

Paso 1: Quitar la palabra function.

```
const sumar = (a, b) { // <- ¡Esto es un error de sintaxis!  
  return a + b;  
};
```

Paso 2: Añadir la "flecha" (=>) entre los parámetros y el cuerpo.

```
const sumar = (a, b) => {  
  return a + b;  
};
```

Anatomía del Código

```
const sumar = (a, b) => {  
  //      ^      ^  
  //      |      |  
  //      Parámetros  Flecha  
  //                  (indica el inicio  
  //                  del cuerpo)  
  return a + b;  
};
```

Las Reglas de la Sintaxis Corta (Optimizaciones):

Las funciones flecha tienen 3 reglas que las hacen aún más cortas.

Regla 1: Si tienes UN SOLO PARÁMETRO, los paréntesis () son opcionales.

```
// Larga  
const duplicar = (n) => {  
  return n * 2;
```

```
// Corta (Regla 1)  
const duplicar = n => {  
  return n * 2;
```

Regla 2: Si NO TIENES PARÁMETROS, los paréntesis () son OBLIGATORIOS.

```
const saludar = () => {  
  return "Hola";  
};
```

Regla 3: El Retorno Implícito (LA MÁS IMPORTANTE).

- Si el cuerpo de tu función (lo que está dentro de {}) contiene **UNA SOLA LÍNEA...**
- ...y esa línea es un return...
- ...puedes **eliminar** las llaves {} y la palabra return.

El valor de esa única línea se retorna automáticamente.

Ejemplo 1 (Reglas 1 y 3):

// Larga

```
const duplicar = (n) => {  
  return n * 2;  
};
```

// Aplicando Regla 3 (Quitar {} y return)

```
const duplicar = (n) => n * 2;
```

// Aplicando Regla 1 (Quitar ())

```
const duplicar = n => n * 2;
```

Anatomía (Retorno Implícito):

```
const duplicar = n => n * 2;
```

```
//      ^      ^
```

```
//      |      |
```

```
//      Parámetro  Expresión que se retorna
```

Ejemplo 2 (con 2 parámetros):

// Larga

```
const sumar = (a, b) => {  
  return a + b;  
};
```

// Aplicando Regla 3 (Retorno Implícito)

```
const sumar = (a, b) => a + b;
```

Ejemplo 3 (sin parámetros):

// Larga

```
const saludar = () => {  
  return "Hola";  
};
```

// Aplicando Regla 3 (Retorno Implícito)

```
const saludar = () => "Hola";
```

Preguntas Frecuentes

¿Significa esto que nunca debo usar `function` y siempre usar `=>`? No. Debes usar la herramienta correcta.

- **Usa `=>` (Flecha)**: En el 90% de los casos, especialmente cuando pasas una función como argumento a otra (en `addEventListener`, `.map()`, `.filter()`, etc)
- **Usa `function` (Regular)**: Cuando necesitas que la función tenga su propio `this` dinámico. El caso más común son los **métodos principales**.

¿Por qué las Expresiones de Función (`const f = ...`) son más seguras que las Declaraciones (`function f...`)?

Por el Hoisting. En un archivo grande, una Declaración de Función puede ser llamada desde cualquier lugar, incluso antes de que la leas, lo cual puede ser confuso. Una Expresión de Función (`const f = ...`) no se puede usar hasta que el código la haya definido. Esto fuerza un orden lógico y previene errores donde se usa una función antes de que esté "lista" (o antes de que el programador mentalmente espere que esté lista).

Ejemplo 1: Funciones Anónimas Tradicionales (Expresiones)

```
// Guardamos una función anónima en la variable 'multiplicar'  
const multiplicar = function(a, b) {  
  return a * b;  
};
```

```
// Guardamos otra en 'dividir'  
const dividir = function(a, b) {  
  if (b === 0) {  
    return "Error: no se puede dividir por cero";  
  }  
  return a / b;  
};
```

```
// Las usamos  
console.log( "Multiplicación (5x7):", multiplicar(5, 7) );  
console.log( "División (10/2):", dividir(10, 2) );  
console.log( "División (10/0):", dividir(10, 0) );
```

Ejemplo 2: Refactorizando a Funciones Flecha (Sintaxis Moderna) Ahora, reescribamos las mismas funciones usando la sintaxis de flecha.

```
const sumar = (a, b) => a + b;  
const restar = (a, b) => a - b;  
const multiplicar = (a, b) => a * b;
```

// 'dividir' necesita un 'if', así que usamos sintaxis larga.

```
const dividir = (a, b) => {  
  if (b === 0) {  
    console.error("¡No se puede dividir por cero!");  
    return null; // O algún valor de error  
  }  
  return a / b;  
};
```

// Probamos una:

```
console.log( sumar(10, 5) ); // Output: 15
```

Ejemplo 3: Pasando una función como argumento Este es el concepto más avanzado, pero podemos hacerlo sin elementos externos. Crearemos una función que recibe otra función como instrucción. Siguiendo el ejemplo anterior:

```
// Esta función es "genérica". No sabe qué operación hará.  
// Acepta dos números y una "operacion" (que esperamos sea una función).
```

```
function ejecutarCalculo(num1, num2, funcionOperacion) {
```

```
  // 1. Recibe los números y la FUNCIÓN.  
  console.log("Ejecutando un cálculo...");
```

```
  // 2. ¡Llama a la función que recibió como argumento!  
  //   y le pasa los números.  
  const resultado = funcionOperacion(num1, num2);
```

```
  // 3. Retorna el resultado.  
  return resultado;  
}
```

Conectamos todo. Ahora llamamos a ejecutarCalculo y le pasamos nuestras funciones-herramienta.

```
// Le pasamos la *variable* que *contiene* la función 'sumar'  
let resultadoSuma = ejecutarCalculo(20, 10, sumar);  
console.log(resultadoSuma); // Output: 30
```

```
// Le pasamos la *variable* 'restar'  
let resultadoResta = ejecutarCalculo(20, 10, restar);  
console.log(resultadoResta); // Output: 10
```

```
// Le pasamos la *variable* 'dividir'  
let resultadoDiv = ejecutarCalculo(20, 0, dividir);  
// Output: ¡No se puede dividir por cero!  
// Output: null
```

Ejemplo demostrativo

Simularemos un sistema de conversión de divisas, pasando de un bloque estructurado a una sintaxis limpia.

```
"use strict";

// Definimos la tasa de cambio
const tasaDolar = 4000;

// 1. Función flecha con bloque completo (llaves y return explícito)
// Se usa cuando hay lógica compleja antes de devolver el valor.
const convertirPesosADolares = (pesos) => {
  if (pesos <= 0) {
    return 0; // Salida temprana si el valor es inválido
  }
  let dolares = pesos / tasaDolar;
  return dolares;
};

// 2. Función flecha con retorno implícito (una sola línea)
// Se usa para cálculos matemáticos directos.
// Noten que 'dolares' no lleva paréntesis porque es un solo parámetro.
const convertirDolaresAPesos = dolares => dolares * tasaDolar;

// --- Simulación de uso con interacción ---
let cantidadPesos = Number(prompt("Ingrese la cantidad de Pesos Colombianos a convertir:"));
let resultadoDolares = convertirPesosADolares(cantidadPesos);
alert("Tus pesos equivalen a: $" + resultadoDolares + " USD");

let cantidadUSD = Number(prompt("Ingrese la cantidad de Dólares a convertir:"));
let resultadoPesos = convertirDolaresAPesos(cantidadUSD);
alert("Tus dólares equivalen a: $" + resultadoPesos + " COP");
```

Práctica Acompañada

Práctica 1: Refactorizando: Convertir funciones tradicionales en funciones flecha modernas.

Paso 1: Observa la función inicial. Es una expresión de función que formatea un saludo.

```
const saludar = function(nombre) {  
  return "¡Hola, " + nombre + "!";  
};
```

```
// Pruébala  
console.log( saludar("Estudiante") );
```

Analiza la función.

- ¿Cuántos parámetros tiene? = Uno, nombre.
- ¿Cuántas líneas de código tiene en su cuerpo? = Una.
- ¿Esa línea es un return? = Respuesta: Sí.

Conclusión: Es un candidato perfecto para una función flecha con retorno implícito y sin paréntesis en el parámetro.

Escribe la versión flecha.

Empieza con `const saludarFlecha =`.

Añade el parámetro (sin paréntesis): `nombre`.

Añade la flecha: `=>`.

Añade la expresión que debe retornar (sin `return` y sin `{}`): `"¡Hola, " + nombre + "!"`.

Trata de cambiarla y usar template literals

```
// La nueva versión Flecha
```

```
const saludarFlecha = nombre => `¡Hola, ${nombre}!`;
```

```
// Comprobación
```

```
console.log( saludarFlecha("Estudiante Flecha") );
```

Troubleshooting - Solución de Errores Comunes

1. Paréntesis olvidados al no tener parámetros:

- Error: `const lanzarDado = => Math.random() * 6;` (SyntaxError)
- Solución: Si la función flecha no recibe ningún parámetro, los paréntesis vacíos son obligatorios: `const lanzarDado = () => ...`

2. Confusión entre bloque de código y retorno implícito:

- Error: `const sumar = (a, b) => { a + b };`
- Explicación: Al colocar llaves {}, anulas el retorno implícito. JavaScript procesa la suma pero no devuelve nada, resultando en undefined.
- Solución: Quita las llaves `const sumar = (a, b) => a + b;` o agrega el return `const sumar = (a, b) => { return a + b; };`

3. Hoisting bloqueado por const:

- Error: Invocar `miFuncion()` en la línea 1 y declararla como `const miFuncion = () => {}` en la línea 5.
- Explicación: A diferencia de function, las variables declaradas con const o let no son inicializadas hasta que el motor de JS llega a esa línea. Dará un error de referencia (ReferenceError).

Actividad de clase

Driver (Piloto): Toma el teclado y escribe la sintaxis.

Navigator (Copiloto): Vigila que no se agreguen paréntesis o llaves innecesarias si se puede usar retorno implícito.

El Reto:

Creen un pequeño script de validación de edad usando interacciones.

1. Creen una función flecha llamada esAdulto que reciba un parámetro edad.
2. La función debe tener un **retorno implícito** que devuelva true si la edad es mayor o igual a 18, o false en caso contrario
Pista: una comparación matemática ya devuelve un booleano.
3. Fuera de la función, usen un prompt para pedir la edad al usuario y guárdenla en una variable (conviértanla a Number).
4. Usen un condicional if llamando a la función esAdulto pasándole la edad.
5. Muestren un alert diciendo "Acceso permitido" o "Acceso denegado".

Tiempo: 20 minutos. Intercambio de roles a la mitad.

Práctica Integradora

Ticket JIRA-114: Refactorización de Saludo

A. Instrucciones: El equipo de Frontend ha dejado esta función regular. Debes refactorizarla para que sea una Función Flecha con **retorno implícito** y con la sintaxis más corta posible.

B. Código Base:

```
function generarSaludo(nombreUsuario) {  
  return "Bienvenido a la plataforma, " + nombreUsuario;  
}  
// Escribe la nueva versión debajo
```

C. Solución Completa:

```
const generarSaludo = nombreUsuario => "Bienvenido a la plataforma, " + nombreUsuario;  
  
console.log(generarSaludo("Ana"));
```

Ejercicio Independiente El Sistema de Salarios

Instrucciones (Ticket JIRA-210 - Módulo de Auditoría Numérica): En la rama feature/auditoria-numeros, el cliente necesita una utilidad que determine si un número ingresado por el usuario es par o impar.

1. Crea una función flecha llamada esPar.
2. Debe recibir un parámetro numero.
3. Debe usar un **retorno implícito** para evaluar si el residuo de la división entre 2 es igual a cero ($\text{numero \% 2} === 0$).
4. Solicita un número al usuario con prompt.
5. Usa la función dentro de un condicional y muestra un alert diciendo "El número es Par" o "El número es Impar".

Criterios de logro esperados:

- La función esPar debe estar escrita en una sola línea.
- Se deben omitir los paréntesis en el parámetro si es uno solo.
- El código debe ser integrado mediante un Pull Request desde la rama feature hacia develop, pasando revisión de pares para asegurar que no se usó sintaxis de función regular.